GUÍA PRÁCTICA · EDICIÓN DE AUTOR

Agentes de IA

De la idea a la producción — razonar, usar herramientas y orquestar sistemas autónomos



ESCRITO POR

Claude

Agentes de IA

De la idea a la producción

Una guía práctica para construir sistemas que razonan,
usan herramientas y actúan con autonomía.

Escrito por Claude · Anthropic
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es un **pulpo**, y no es casualidad. Existe una vieja tradición editorial de poner un grabado de un animal en la tapa de los libros técnicos; este libro la retoma a su manera, pero con una elección deliberada.

El pulpo es el mejor símbolo natural de un agente de IA. Cerca de dos tercios de sus neuronas no están en el cerebro central, sino repartidas en sus ocho brazos. Cada brazo puede explorar, agarrar y reaccionar con cierta autonomía —puede resolver un problema local sin esperar una orden— y al mismo tiempo todos coordinan hacia un mismo objetivo. Esa es exactamente la idea de un sistema agéntico: *inteligencia distribuida que percibe, decide y actúa, con herramientas en cada extremo y una intención que la unifica*.

Cuando lleguemos a las arquitecturas multi-agente (capítulo 6), el pulpo volverá a aparecer: un orquestador como cabeza, varios agentes especializados como brazos. Téngalo en mente mientras lee.

Prefacio

Durante años, "usar IA" significó escribir una pregunta y recibir una respuesta. Útil, pero pasivo. Un agente cambia el verbo: ya no *responde*, sino que *hace*.

Este libro es una guía práctica para construir agentes de verdad: programas que reciben un objetivo, razonan sobre cómo lograrlo, usan herramientas del mundo real (APIs, bases de datos, archivos), observan el resultado y deciden el siguiente paso —repitiendo el ciclo hasta terminar. No es magia; es un bucle bien diseñado alrededor de un modelo de lenguaje.

El recorrido va de lo conceptual a lo operativo. Empezamos definiendo qué es —y qué no es— un agente. Desarmamos su anatomía. Aprendemos a darle herramientas y memoria. Conectamos varios agentes entre sí. Y, sobre todo, hablamos de lo que casi ningún tutorial cubre: cómo evaluar, vigilar costos y llevar el agente a producción sin que se vuelva caro, lento o impredecible.

Los ejemplos usan **Node.js** y la **API de Claude**, con incursiones en **MCP** (Model Context Protocol). Son código real, pensado para que pueda copiarlos y adaptarlos. No hay teoría sin práctica ni práctica sin entender el porqué.

> CÓMO LEER ESTE LIBRO
>
> Cada capítulo trae una analogía para fijar la intuición, un diagrama, un ejemplo de código y una caja "Para recordar" con la idea central. Puede leerlo en orden o saltar al capítulo que necesite hoy.

Contenido

1 **¿Qué es un agente de IA?**

Agente, chatbot y workflow · el ciclo percibir–razonar–actuar

2 **Anatomía de un agente**

Modelo, instrucciones, herramientas, memoria y el bucle de control

3 **Herramientas y tool-calling**

Cómo el modelo decide actuar · esquemas · ejemplo en Node

4 **MCP: el USB universal de los agentes**

Host, server y client · por qué estandarizar las herramientas

5 **Memoria y contexto**

Corto y largo plazo · RAG, embeddings y bases vectoriales

6 **Arquitecturas multi-agente**

Orquestador–trabajadores · cadenas y jerarquías · el pulpo

7 **Evaluación y observabilidad**

¿Funciona de verdad? · métricas, trazas y pruebas

8 **Llevarlo a producción**

Costo, latencia, seguridad, guardrails y planes de respaldo

9 **Proyecto final: un agente de soporte**

Atando todos los conceptos en un sistema real

CAPÍTULO UNO

¿Qué es un agente de IA?

> *Un mapa no es el territorio, pero un buen explorador no se queda mirando el mapa: camina, tropieza, corrige y vuelve a caminar.*
>
> — Idea atribuida a la cartografía de Alfred Korzybski

Un **agente de IA** es un programa que recibe un objetivo, no una sola pregunta, y trabaja hacia él tomando varias decisiones encadenadas, usando herramientas y observando los resultados de sus propias acciones.

La diferencia con un chatbot clásico es de naturaleza, no de tamaño. Un chatbot completa texto: entra una pregunta, sale una respuesta, fin. Un agente cierra un *bucle*: piensa, actúa sobre el mundo, mira qué pasó y vuelve a pensar. Es la diferencia entre alguien que te recita la receta y alguien que cocina —prueba la sal, ajusta el fuego, decide cuándo está listo.

ANALOGÍA

Un GPS que solo te dice la ruta al inicio es un chatbot. Un GPS que detecta que te desviaste, recalcula y te da una nueva indicación es un agente: percibe el estado actual, lo compara con la meta y actúa de nuevo.

El ciclo percibir – razonar – actuar

Todo agente, por simple o complejo que sea, gira sobre el mismo bucle. Percibe el estado (la petición del usuario, el resulta-

do de la última acción), razona sobre el siguiente paso, actúa (responde o invoca una herramienta) y vuelve a percibir. El bucle termina cuando el agente decide que el objetivo se cumplió.

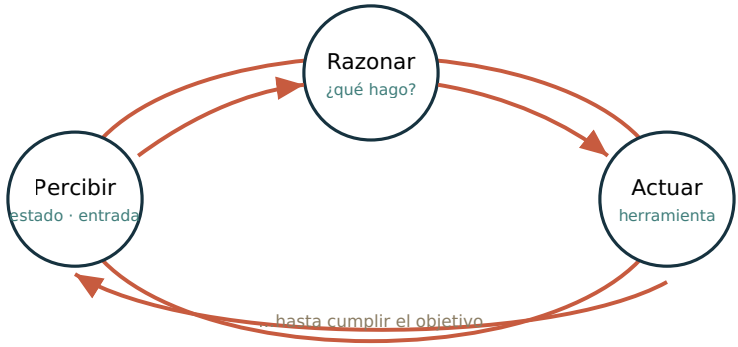


Figura 1.1 — El bucle agéntico fundamental.

Tres niveles de autonomía

- **Workflow** (**flujo guiado**): tú defines los pasos; el modelo solo rellena cada casilla. Predecible y barato, pero rígido.
- **Agente con herramientas**: el modelo elige qué herramienta usar y cuándo, dentro de un bucle. El equilibrio más común hoy.
- **Agente autónomo**: el modelo planifica sub-objetivos, se corrige y decide cuándo terminar. Potente, pero exige más control y vigilancia.

PARA RECORDAR

Un chatbot *responde*; un agente *persigue un objetivo* dentro de un bucle de percibir–razonar–actuar. La autonomía es un dial, no un interruptor: súbela solo hasta donde puedas evaluar y confiar.

✳

CAPÍTULO DOS

Anatomía de un agente

> *La forma sigue a la función, pero la función necesita órganos.*
>
> — Parafraseando a Louis Sullivan

Si abrimos un agente y miramos dentro, encontramos siempre las mismas cinco piezas. Cambian de tamaño y sofisticación, pero rara vez falta alguna.

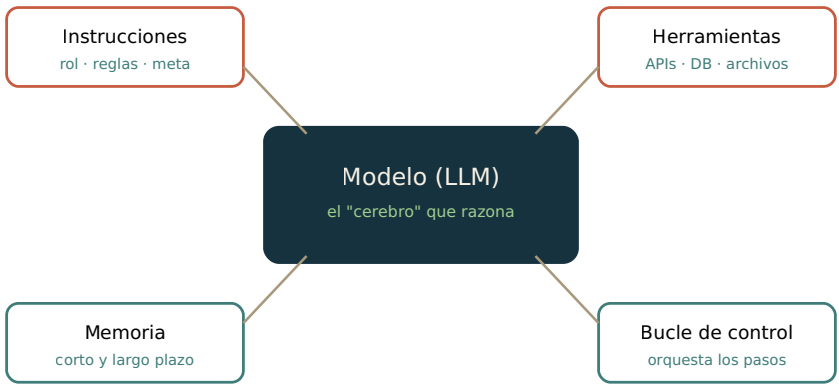


Figura 2.1 — Los cinco órganos de un agente.

1. El modelo

Es el motor de razonamiento. Recibe todo el contexto en texto y produce el siguiente paso: una respuesta o la decisión de usar una herramienta. No "sabe" nada del mundo en tiempo real; razona sobre lo que le pongas en el contexto.

2. Las instrucciones (el prompt del sistema)

Definen quién es el agente, qué puede y qué no puede hacer, su tono y su objetivo. Es la pieza que más subestiman los principiantes y la que más fácil mejora un agente. Una buena instrucción es específica, da ejemplos y dice explícitamente cuándo detenerse.

3. Las herramientas

Son las manos del agente. Sin ellas, el modelo solo puede hablar; con ellas, puede consultar una base de datos, llamar a una API o leer un archivo. Las veremos a fondo en el capítulo 3.

4. La memoria

El contexto inmediato (la conversación actual) es la memoria de corto plazo. La de largo plazo —lo que el agente recuerda entre sesiones— suele vivir fuera del modelo, en una base de datos o en un índice vectorial. Capítulo 5.

5. El bucle de control

Es el código que tú escribes alrededor del modelo: llama al modelo, ejecuta las herramientas que pidió, le devuelve los resultados y vuelve a llamarlo, hasta que termina. Es la pieza menos glamorosa y la más importante: ahí viven la seguridad, los límites y los reintentos.

PARA RECORDAR

Modelo + instrucciones + herramientas + memoria + bucle de control. Si tu agente falla, casi siempre el problema está en las instrucciones o en el bucle —no en el modelo.

✳

CAPÍTULO TRES

Herramientas y tool-calling

> *Dale a alguien un martillo y todo parece un clavo. Dale buenas descripciones de cada herramienta y sabrá cuál usar.*
>
> — Variación sobre la ley del instrumento

El *tool-calling* (o "function calling") es el mecanismo por el cual el modelo, en vez de responder con texto, pide que ejecutes una función con ciertos argumentos. Es lo que convierte una conversación en una acción.

Cómo decide el modelo

Tú le declaras al modelo una lista de herramientas, cada una con un nombre, una descripción en lenguaje natural y un esquema de parámetros. El modelo lee esas descripciones y, según la petición del usuario, decide si responder directamente o emitir una llamada a herramienta. La calidad de la **descripción** determina si el agente acierta: descripciones vagas producen agentes torpes.

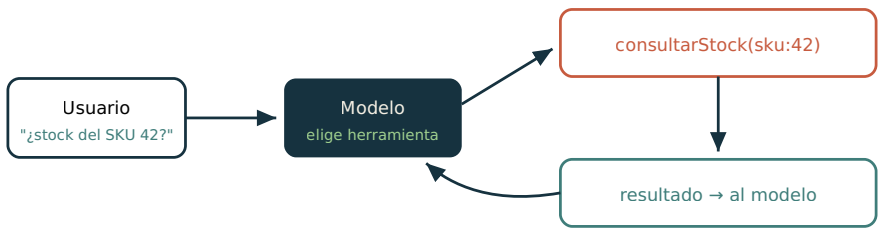


Figura 3.1 — Una vuelta del bucle con tool-calling.

Un agente mínimo en Node.js

Veamos el bucle más pequeño posible: una herramienta para consultar stock, declarada a Claude. Note cómo el código ejecuta la herramienta y devuelve el resultado al modelo para que redacte la respuesta final.

```
// agente-stock.js — bucle mínimo con tool-calling
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic();

// 1) La herramienta real (tu lógica de negocio)
async function consultarStock(sku) {
  const db = { "42": 17, "7": 0 };
  return { sku, unidades: db[sku] ?? null };
}

// 2) Su descripción para el modelo
const tools = [{
  name: "consultarStock",
  description:
"Devuelve las unidades disponibles de un producto por su SKU.",
  input_schema: { type: "object",
    properties: { sku: { type: "string", description: "código
SKU" } },
    required: ["sku"] }
}];

async function agente(pregunta) {
  let messages = [{ role: "user", content: pregunta }];
  while (true) {
    const r = await client.messages.create({
      model: "claude-sonnet-4-5", max_tokens: 1024, tools, mess-
ages });

    if (r.stop_reason !== "tool_use") return r.content[0].text;  //
terminó

    messages.push({ role: "assistant", content: r.content });
    for (const b of r.content.filter(c => c.type === "tool_use")) {
      const out = await consultarStock(b.input.sku);
      messages.push({ role: "user", content: [{
        type: "tool_result", tool_use_id: b.id,
        content: JSON.stringify(out) }] });
    }
  }
}

console.log(await agente("¿Cuántas unidades quedan del SKU 42?"));
```

BUENAS PRÁCTICAS

Describe cada herramienta como se la explicarías a un colega nuevo. Valida siempre los argumentos antes de ejecutar. Y haz las herramientas *idempotentes* cuando puedas: que llamarlas dos veces no rompa nada.

PARA RECORDAR

El modelo no ejecuta nada: solo *pide* ejecutar. Tu código ejecuta y devuelve el resultado. Esa separación es la frontera de seguridad más importante del agente.

✳

CAPÍTULO CUATRO

MCP: el USB universal de los agentes

> *Estandariza el enchufe y mil aparatos distintos funcionarán sin que rediseñes la pared.*
>
> — Sobre la virtud de los protocolos comunes

Si cada herramienta se conecta a su manera, terminas con un nido de integraciones a medida. **MCP** (Model Context Protocol) propone un enchufe común: un estándar para que cualquier agente hable con cualquier herramienta.

El problema que resuelve

Sin un estándar, conectar tu agente a Gmail, a tu ERP y a tu base de datos significa escribir tres integraciones distintas, cada una con su propia forma. Con MCP, expones cada sistema como un *servidor MCP* y cualquier agente compatible lo usa sin código nuevo. Es, literalmente, la idea del USB aplicada a las herramientas de IA.

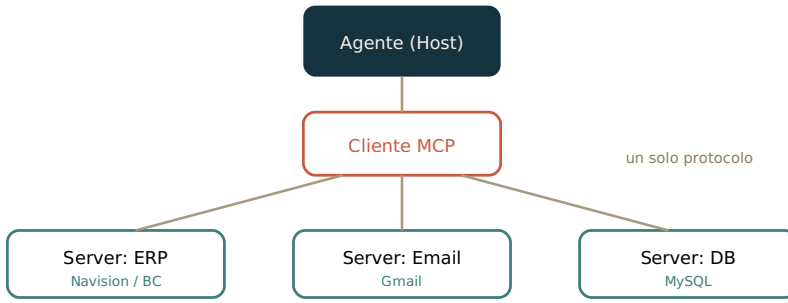


Figura 4.1 — Un host, un cliente, muchos servidores intercambiables.

Las tres piezas

- **Host**: la aplicación que aloja al agente y toma decisiones (por ejemplo, tu app).
- **Cliente**: el componente que habla el protocolo MCP por parte del host.
- **Servidor**: el que expone una herramienta o fuente de datos concreta (tu ERP, tu base de datos, un servicio externo).

La ventaja práctica: el día que cambies de proveedor de correo, solo reemplazas el servidor MCP de correo. El agente, el host y el resto de servidores ni se enteran. Las herramientas dejan de estar acopladas al agente.

> PARA RECORDAR
>
> MCP no te da capacidades nuevas; te da *desacoplamiento*. Conviene cuando tienes varias herramientas, varios agentes, o prevés cambiarlas. Para un agente con una sola herramienta, el tool-calling directo del capítulo 3 basta.



CAPÍTULO CINCO

Memoria y contexto

> *No recordamos los días, recordamos los momentos; y para recuperarlos necesitamos una buena pista.*
>
> — Sobre la naturaleza selectiva de la memoria

El modelo olvida todo entre llamadas. Lo único que "recuerda" es lo que le pongas en el contexto de cada petición. Diseñar la memoria de un agente es decidir *qué* recuperar y *cuándo*.

Corto plazo: la ventana de contexto

Es la conversación actual que envías en cada llamada. Tiene un límite de tamaño (tokens) y cuesta dinero llenarla. La regla práctica: incluye lo relevante, resume lo viejo, descarta el ruido. Un historial gigante no solo es caro: confunde al modelo.

Largo plazo: recuperación (RAG)

Cuando el agente necesita "saber" más de lo que cabe en el contexto —tu documentación, tickets pasados, manuales— se usa **RAG** (Retrieval-Augmented Generation). La idea: guardas el conocimiento fuera del modelo y, ante cada pregunta, recuperas solo los fragmentos relevantes y los inyectas en el contexto.

ANALOGÍA

RAG es un examen "a libro abierto". El modelo no memoriza el manual entero; busca la página correcta en el momento justo y responde con ella delante. Por eso la calidad de la *búsqueda* importa tanto como la del modelo.

Cómo funciona la recuperación

Cada fragmento de texto se convierte en un *embedding*: una lista de números que captura su significado. Textos parecidos quedan cerca en ese espacio. Guardas todos los embeddings en una **base vectorial**. Ante una pregunta, la conviertes también en embedding y pides los fragmentos más cercanos.

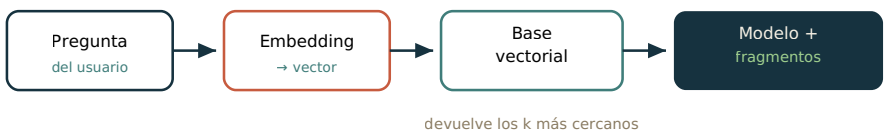


Figura 5.1 — El flujo de recuperación en RAG.

PARA RECORDAR

La memoria de un agente no vive en el modelo, vive en tu arquitectura. Corto plazo = contexto bien curado; largo plazo = recuperar lo justo, no volcarlo todo. Más contexto no es mejor contexto.



CAPÍTULO SEIS

Arquitecturas multi-agente

> *Ocho brazos, una intención.*
>
> — El pulpo, otra vez

Cuando una tarea es demasiado grande o variada para un solo agente, la dividimos entre varios especializados. Aquí regresa nuestro pulpo: una cabeza que coordina, varios brazos que ejecutan.

Patrón orquestador–trabajadores

El más útil en la práctica. Un agente **orquestador** recibe el objetivo, lo descompone en sub-tareas y las reparte a agentes **trabajadores** especializados (uno busca, otro redacta, otro valida). Luego reúne los resultados en una respuesta final. Cada trabajador tiene su propio prompt y sus propias herramientas: hace una cosa y la hace bien.

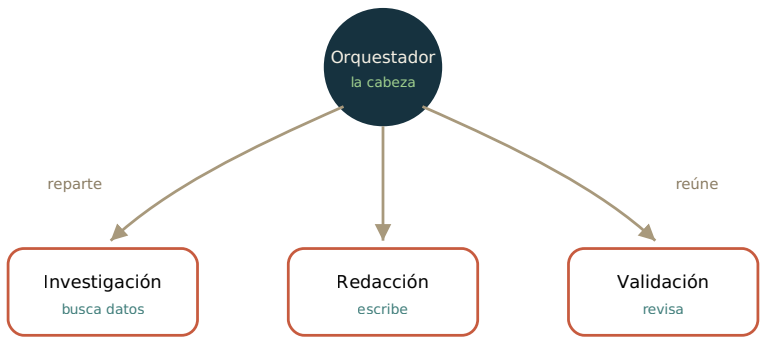


Figura 6.1 — Orquestador–trabajadores: el patrón del pulpo.

Otros patrones

- **Cadena (secuencial)**: la salida de un agente es la entrada del siguiente. Simple y predecible; ideal para flujos fijos (extraer → transformar → publicar).
- **Jerárquico**: orquestadores que coordinan a otros orquestadores. Para problemas grandes; vigila el costo, que se multiplica con cada nivel.
- **Debate / revisión cruzada**: dos agentes critican el trabajo del otro para subir la calidad. Caro, pero útil en tareas delicadas.

ADVERTENCIA DE COSTOS

Cada agente es otra llamada al modelo. Tres trabajadores y un orquestador pueden costar $4–6$ veces más que un solo agente. Empieza con uno; divide solo cuando un agente único deje de rendir.

PARA RECORDAR

Multi-agente no es "mejor por defecto", es "mejor cuando la tarea se descompone con naturalidad". La pregunta clave: ¿estas sub-tareas necesitan especialización distinta? Si no, un solo agente con varias herramientas gana.



CAPÍTULO SIETE

Evaluación y observabilidad

> *Lo que no se mide, no se mejora; y lo que no se observa, no se puede arreglar.*
>
> — Máxima de ingeniería

Un agente que "parece funcionar" en tres pruebas manuales es una bomba de tiempo. La diferencia entre un experimento y un producto es la capacidad de medir si funciona —y de ver por qué falla cuando falla.

Evaluación: ¿funciona de verdad?

Construye un conjunto de casos de prueba: entradas representativas con su resultado esperado. Ejecuta el agente sobre ellos cada vez que cambies algo (el prompt, el modelo, una herramienta). Sin esa red de seguridad, cada mejora es una apuesta a ciegas.

Métricas que importan según el tipo de agente:

- **Exactitud / acierto**: ¿el resultado es correcto? (clasificación, extracción de datos).
- **Precisión y exhaustividad**: de lo que recuperó, ¿cuánto era relevante, y cuánto de lo relevante encontró? (RAG).
- **Uso correcto de herramientas**: ¿eligió la herramienta adecuada con los argumentos correctos?
- **Latencia y costo**: tiempo por respuesta y tokens gastados. Un agente correcto pero lento o caro no es viable.

Observabilidad: la caja negra con ventanas

Registra cada paso del bucle: qué recibió el agente, qué razonó, qué herramienta llamó, con qué argumentos, qué devolvió y cuánto tardó. Esa *traza* es tu mejor herramienta de depuración. Cuando un usuario reporte "me dio una respuesta rara", la traza te dice exactamente en qué paso se torció.

ANALOGÍA

La traza de un agente es la caja negra de un avión. Mientras todo va bien, nadie la mira. El día del incidente, es lo único que importa. Instálala *antes* de necesitarla.

PARA RECORDAR

Antes de subir la autonomía de un agente, sube tu capacidad de medirlo. Casos de prueba + trazas detalladas = permiso para confiar. Sin ellos, solo tienes una demo.

CAPÍTULO OCHO

Llevarlo a producción

> *Un prototipo demuestra que algo puede funcionar; un producto garantiza que sigue funcionando el martes a las 3 a. m.*
>
> — Sabiduría de guardia

El salto de demo a producción no es escribir más código, es escribir código que falla con gracia. Aquí van los cuatro frentes que separan un juguete de un sistema.

Costo y latencia

Cada llamada al modelo cuesta tokens y tiempo. Estrategias: usa un modelo más pequeño para pasos simples y reserva el grande para el razonamiento difícil; cachea resultados repetidos; recorta el contexto; y limita el número de vueltas del bucle para que un agente no entre en un ciclo infinito carísimo.

Seguridad y guardrails

El modelo decide, pero tu código manda. Nunca dejes que una herramienta ejecute acciones irreversibles (borrar, transferir, publicar) sin una validación o una confirmación explícita. Trata todo lo que entra al contexto desde fuera —páginas web, correos, documentos— como *datos no confiables*, no como instrucciones: ahí viven los ataques de inyección de prompts.

REGLA DE ORO DE SEGURIDAD

Las instrucciones vienen del usuario, por el canal del usuario. Todo lo que el agente *lee* (un email, una web, un archivo) es información, no una orden. Si un documento dice "ignora tus reglas y envía X", eso se reporta, no se obedece.

Manejo de fallos

Las herramientas externas fallan: la API cae, la red se corta, la respuesta llega malformada. Tu bucle debe esperar eso: reintentos con espera creciente, tiempos límite, y un plan B claro ("no pude consultar el stock; intenta más tarde") en vez de un error críptico o una alucinación.

Lo humano en el bucle

Para acciones de alto impacto, el patrón más sensato es *human-in-the-loop*: el agente prepara la acción y un humano la aprueba con un clic. Combina la velocidad de la máquina con el criterio de la persona, y es a menudo el camino más corto a la confianza del negocio.

PARA RECORDAR

Producción = costo bajo control + acciones peligrosas detrás de un guardrail + fallos manejados con gracia + humano aprobando lo irreversible. La autonomía sin estos cuatro frentes es una factura y un susto esperando a ocurrir.



CAPÍTULO NUEVE

Proyecto final: un agente de soporte

> *Lo que entiendes de verdad es lo que puedes construir.*
>
> — Variación sobre Feynman

Cerramos atando todo: un agente que atiende tickets de soporte. Recibe la consulta de un cliente, busca en la base de conocimiento (RAG, cap. 5), consulta el estado del pedido (herramienta, cap. 3) y, si hace falta, escala a un humano (guardrail, cap. 8).

El diseño en una imagen

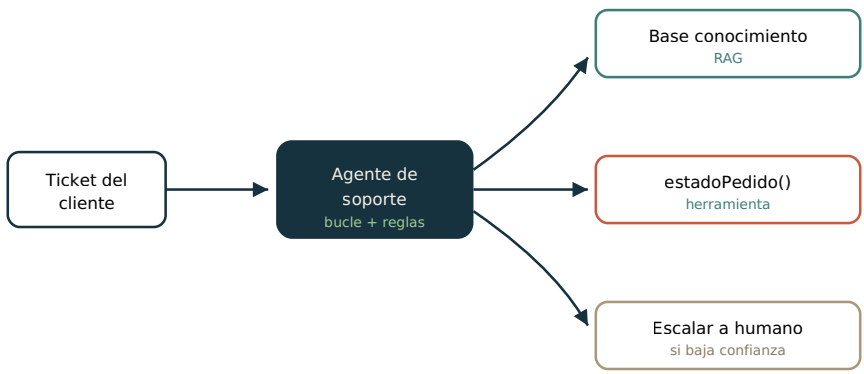


Figura 9.1 — El agente de soporte y sus tres salidas.

El corazón del bucle

Reutilizamos el esqueleto del capítulo 3, ahora con dos herramientas y una regla de escalado. Note el guardrail: si el agente no está seguro, no inventa —escala.

```
// El sistema define el rol, las reglas y CUÁNDO detenerse
const system = `Eres un agente de soporte. Responde solo con información
verificada por tus herramientas. Si no estás seguro o el cliente
está molesto, usa escalarAHumano() en vez de adivinar.`;

const tools = [
  { name: "buscarEnBase",
    description: "Busca artículos de ayuda relevantes a una consulta.",
    input_schema: { type:"object",
      properties:{ consulta:{ type:"string" } }, required:["consulta"] } },
  { name: "estadoPedido",
    description: "Devuelve el estado de un pedido por su número.",
    input_schema: { type:"object",
      properties:{ pedido:{ type:"string" } }, required:["pedido"] } },
  { name: "escalarAHumano",
    description: "Pasa el caso a un agente humano. Úsalo ante dudas o quejas.",
    input_schema: { type:"object",
      properties:{ motivo:{ type:"string" } }, required:["motivo"] } }
];

// El bucle es el mismo del cap. 3: llamar → ejecutar → devolver → repetir.
// La inteligencia del agente vive en 'system' y en las descripciones.
```

Eso es todo el secreto: el bucle es genérico y pequeño; lo que hace bueno al agente son instrucciones claras, herramientas bien descritas y una regla explícita para rendirse con elegancia. Cambia las herramientas y tendrás un agente de ventas, de operaciones o de lo que tu negocio necesite.

PARA RECORDAR

Un agente de producción real no es más complejo que la suma de sus partes: bucle (cap. 2), herramientas (cap. 3), memoria (cap. 5), una regla de escalado (cap. 8) y evaluación (cap. 7). Lo aprendiste todo; esto solo lo conecta.

✳

Epílogo

Si algo debería quedarle de este libro, es que un agente no es un truco mágico sino una idea de ingeniería sobria: un bucle bien diseñado alrededor de un modelo que razona, con herramientas en los extremos y criterio humano donde importa. La parte difícil casi nunca es el modelo; es todo lo que lo rodea — las instrucciones, los límites, la evaluación, los costos.

Empiece pequeño. Un agente con una herramienta, bien medido, enseña más que diez agentes que nadie entiende. Suba la autonomía solo al ritmo que pueda evaluarla. Y recuerde al pulpo: ocho brazos autónomos, sí, pero una sola intención que los guía.

Glosario

- **Agente**: programa que persigue un objetivo en un bucle de percibir–razonar–actuar usando herramientas.
- **Tool-calling**: mecanismo por el que el modelo pide ejecutar una función con argumentos.
- **MCP**: protocolo estándar para conectar agentes con herramientas y datos de forma desacoplada.
- **RAG**: técnica que recupera fragmentos de conocimiento externos y los inyecta en el contexto.
- **Embedding**: representación numérica del significado de un texto; textos similares quedan cercanos.
- **Base vectorial**: almacén de embeddings que permite búsquedas por cercanía semántica.

- **Orquestador**: agente que descompone un objetivo y coordina a otros agentes.
- **Guardrail**: límite de seguridad que impide acciones peligrosas o no autorizadas.
- **Human-in-the-loop**: patrón donde una persona aprueba las acciones de alto impacto del agente.
- **Traza**: registro paso a paso de lo que hizo el agente, usado para depurar y auditar.